

Module 3: Case Study - 1

Problem Statement:

You work for XYZ Corporation that uses on premise solutions and a limited number of systems. With the increase in requests in their application, the load also increases. So, to handle the load the corporation has to buy more systems almost on a regular basis. Realizing the need to cut down the expenses on systems, they decided to move their infrastructure to AWS.

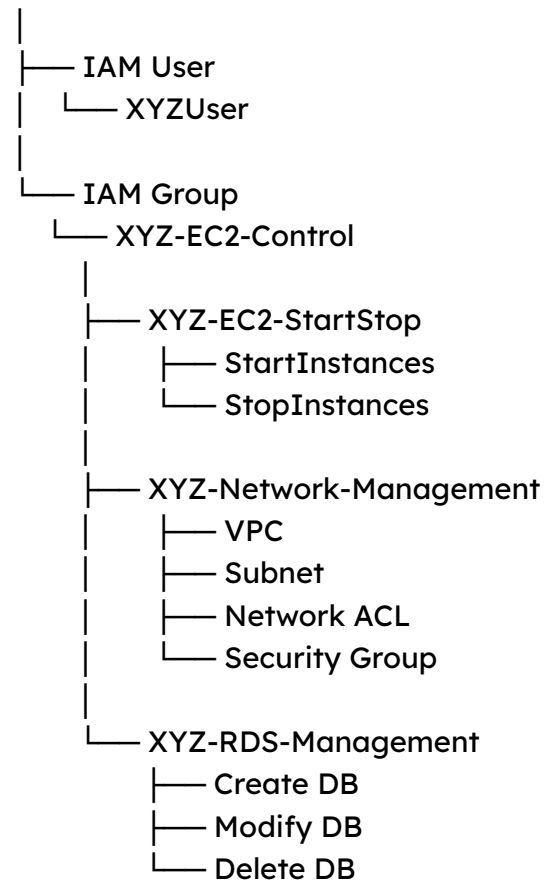
Tasks Performed:

1.
 - a. To create a user account that can login to the console
 - b. To create a group and make sure that the group can only launch and to stop EC2 instances using that previously created account
2.
 - a. To provide permission to let the user of a previously created account to create VPCs, Subnets, NACL and security groups
 - b. further, to add the permission so that the user can create RDS instance
 - c. To explore security options to protect the AWS resources and to secure the permissions provided to the group.

Critical Thinking:

Architecture to build:

AWS Account



Task 1 — IAM User and EC2 Start/Stop Group

A. Created an IAM user

AWS Console → IAM → Users → Create user

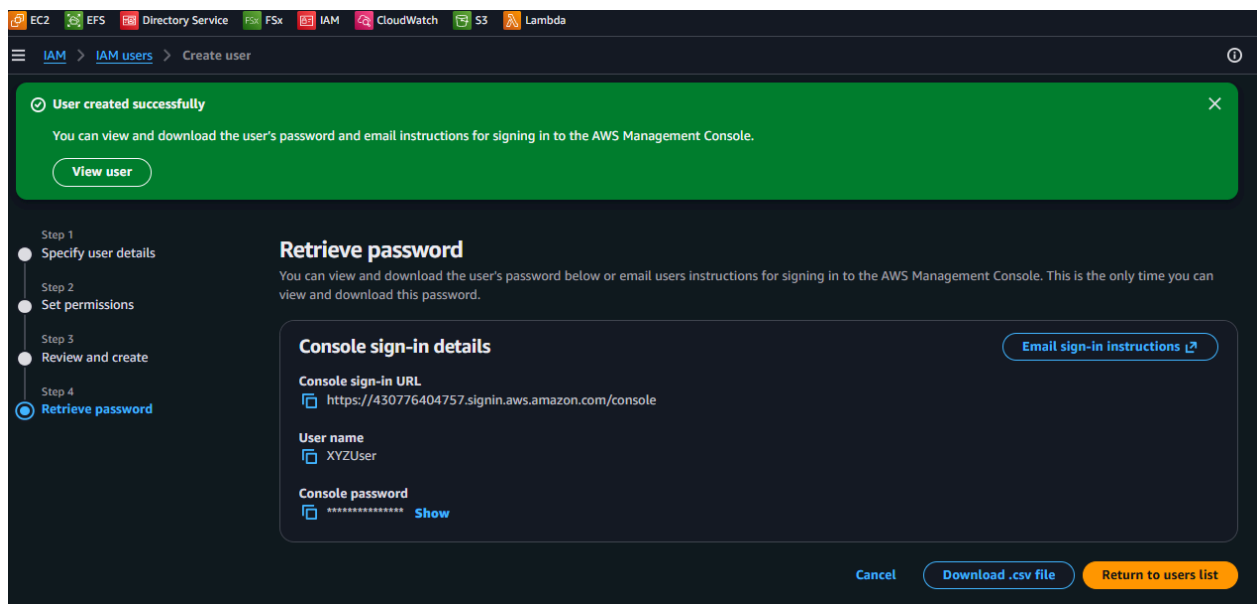
User name: XYZUser

Selected: Provide user access to the AWS Management Console

Chosen an appropriate password: 123Realme

Clicked Create user.

Successfully created a user: XYZUser



B. Create the EC2 Start/Stop Policy

IAM → Policies → Create policy → JSON

Used: Amazon Q to create JSON version for the policy.

Script:

```
{  
  "Version": "2012-10-17",
```

```

"Statement": [
  {
    "Sid": "AllowStartStopEC2Instances",
    "Effect": "Allow",
    "Action": [
      "ec2:StartInstances",
      "ec2:StopInstances"
    ],
    "Resource": "arn:aws:ec2:*:*:instance/*"
  },
  {
    "Sid": "AllowDescribeInstances",
    "Effect": "Allow",
    "Action": [
      "ec2:DescribeInstances",
      "ec2:DescribeInstanceStatus"
    ],
    "Resource": "*"
  }
]
}

```

Policy name: XYZ-EC2-StartStop

Created the policy.

The screenshot shows the AWS IAM console interface. At the top, a green banner indicates 'Policy XYZ-EC2-StartStop created.' with a 'View policy' button. Below this, the policy name 'XYZ-EC2-StartStop' is displayed with an 'Info' link and 'Edit' and 'Delete' buttons. The 'Policy details' section shows the following information:

Policy details	Creation time	Edited time	ARN
Type Customer managed	September 26, 2026, 11:39 (UTC+05:30)	September 26, 2026, 11:39 (UTC+05:30)	arn:aws:iam::430776404757:policy/XYZ-EC2-StartStop

Below the details, there are tabs for 'Permissions', 'Entities attached', 'Tags', 'Policy versions (1)', and 'Last Accessed'. The 'Permissions' tab is selected, showing 'Permissions defined in this policy' with an 'Info' link and 'Edit', 'Summary', and 'JSON' buttons. A search bar is present. The permissions list shows 'Allow (1 of 476 services)' with a toggle for 'Show remaining 475 services'. The table below lists the permissions:

Service	Access level	Resource	Request condition
EC2	Limited: List, Write	Multiple	None

C. Create an IAM Group

IAM → User groups → Create group

Group name: XYZ-EC2-Control

Added User: XYZUser

Attach the XYZ-EC2-StartStop policy

The screenshot shows the 'Create user group' page in the AWS IAM console. The breadcrumb navigation is 'IAM > IAM user groups > Create user group'. The page title is 'Create user group'. There are three main sections:

- Name the group:** A text input field for 'User group name' containing 'XYZ-EC2-Control'. A note below says 'Enter a meaningful name to identify this group.' and 'Maximum 128 characters. Use alphanumeric and '+', '@', '-' characters.'
- Add users to the group - Optional (1/1):** A table with columns: 'User name', 'Group', 'Last activity', and 'Creation time'. One user, 'XYZUser', is listed with 0 groups and last activity 39 minutes ago.
- Attach permissions policies - Optional (1/1224):** A table with columns: 'Policy name', 'Type', 'Used as', and 'Description'. One policy, 'XYZ-EC2-StartStop', is listed with type 'Customer managed' and 'Used as' 'None'.

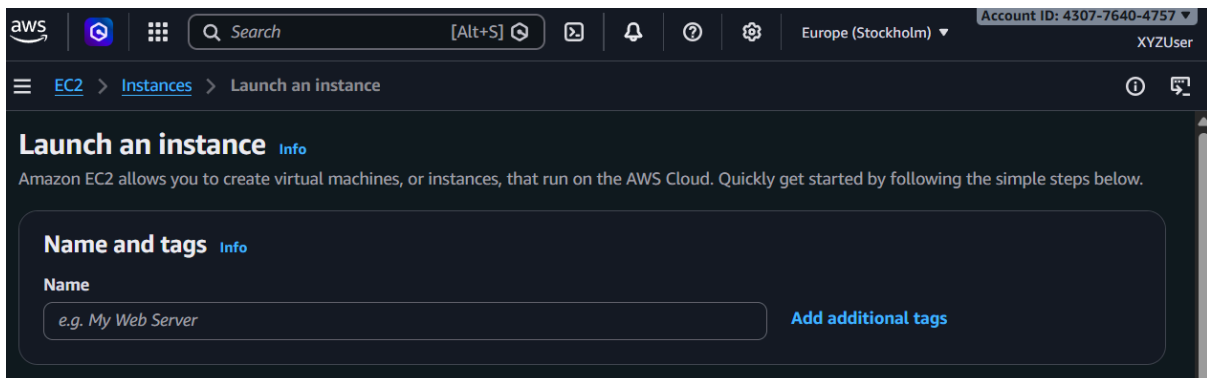
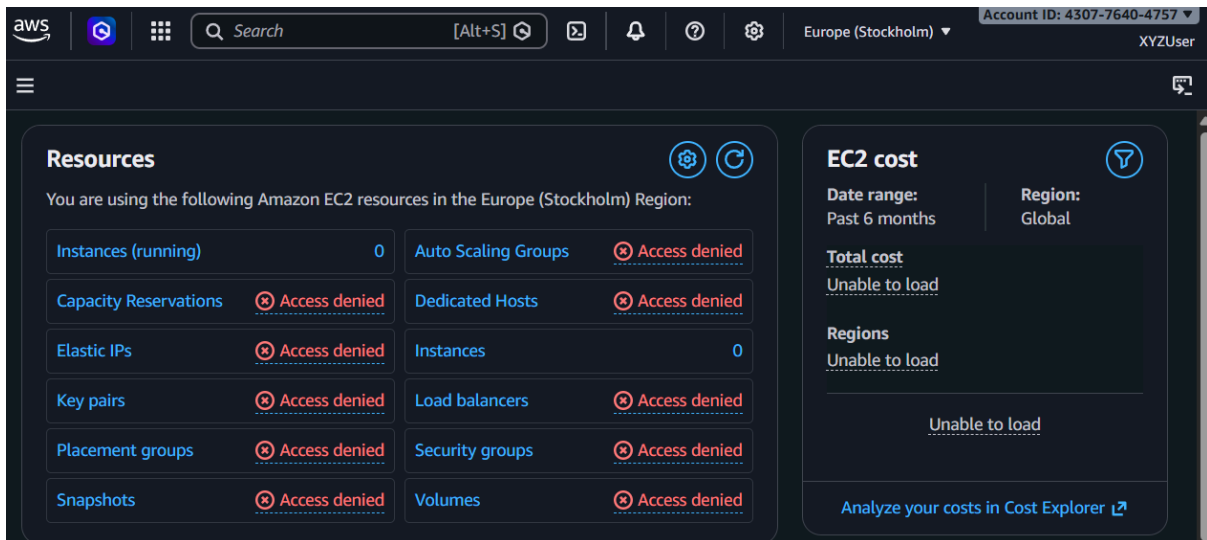
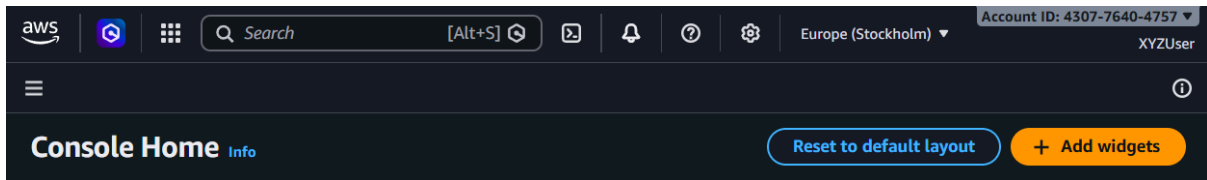
At the bottom right, there are 'Cancel' and 'Create user group' buttons.

The screenshot shows the 'IAM user groups' page in the AWS IAM console. The breadcrumb navigation is 'IAM > IAM user groups'. A green success message at the top says 'XYZ-EC2-Control user group created.' with a 'View group' button. Below the message, the page title is 'IAM user groups (1)' and there are 'Delete' and 'Create group' buttons. A table lists the user groups:

Group name	Users	Permissions	Creation time
XYZ-EC2-Control	1	Defined	Now

D. Tested

Logged in using XYZUser.



The user is only allowed to: Start EC2 instances and Stop EC2 instances

The user should not be able to:

- Terminate EC2 instances
- Create EC2 instances
- Modify security groups
- Create VPCs
- Create RDS instances

This demonstrates the principle of least privilege.

Task 2: Attaching additional Policies

1. VPC, Subnet, NACL and Security Group Permissions

To extend the user's permissions, created another policy named XYZ-Network-Management which will give rights to the user - XYZuser to create VPC, Subnet, NACL and Security Group.

Used Amazon Q to get JSON script

Named: XYZ-Network-Management

Script used:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "VPCManagement",
      "Effect": "Allow",
      "Action": [
        "ec2:CreateVpc",
        "ec2:DeleteVpc",
        "ec2:DescribeVpcs",
        "ec2:ModifyVpcAttribute"
      ],
      "Resource": "*"
    },
    {
      "Sid": "SubnetManagement",
      "Effect": "Allow",
      "Action": [
        "ec2:CreateSubnet",
        "ec2:DeleteSubnet",
        "ec2:DescribeSubnets",
        "ec2:ModifySubnetAttribute"
      ],
      "Resource": "*"
    },
    {
      "Sid": "NACLManagement",
      "Effect": "Allow",
      "Action": [
```

```

        "ec2:CreateNetworkAcl",
        "ec2>DeleteNetworkAcl",
        "ec2:CreateNetworkAclEntry",
        "ec2>DeleteNetworkAclEntry",
        "ec2:ReplaceNetworkAclAssociation",
        "ec2:DescribeNetworkAcls"
    ],
    "Resource": "*"
},
{
    "Sid": "SecurityGroupManagement",
    "Effect": "Allow",
    "Action": [
        "ec2:CreateSecurityGroup",
        "ec2>DeleteSecurityGroup",
        "ec2:AuthorizeSecurityGroupIngress",
        "ec2:AuthorizeSecurityGroupEgress",
        "ec2:RevokeSecurityGroupIngress",
        "ec2:RevokeSecurityGroupEgress",
        "ec2:DescribeSecurityGroups"
    ],
    "Resource": "*"
}
]
}

```

IAM
>
Policies
>
XYZ-Network-Management

Policy XYZ-Network-Management created.
View policy

XYZ-Network-Management

give rights to the user - XYZuser to create VPC, Subnet, NACL and Security Group

Edit
Delete

Policy details

Type	Creation time	Edited time	ARN
Customer managed	September 26, 2026, 12:04 (UTC+05:30)	September 26, 2026, 12:04 (UTC+05:30)	arn:aws:iam::430776404757:policy/XYZ-Network-Management

Permissions

Entities attached

Tags

Policy versions (1)

Last Accessed

Permissions defined in this policy

Permissions defined in this policy document specify which actions are allowed or denied. To define permissions for an IAM identity (user, user group, or role), attach a policy to it

(1)

Search

Allow (1 of 476 services)

Show remaining 475 services

Service	Access level	Resource	Request condition
EC2	Limited: List, Write, Tagging	All resources	ec2:CreateAction = CreateVpc,CreateSubnet,CreateNetworkAcl,CreateSecurityGroup

Attach the policy to: XYZ-EC2-Control

Summary

User group name	Creation time	ARN
XYZ-EC2-Control	September 26, 2026, 11:50 (UTC+05:30)	arn:aws:iam::430776404757:group/XYZ-EC2-Control

Users (1) | **Permissions** | Access Advisor

Permissions policies (2) Info

You can attach up to 10 managed policies.

Search Filter by Type: All types

☐	Policy name ↗	Type	Attached entities
☐	XYZ-EC2-StartStop	Customer managed	1
☐	XYZ-Network-Management	Customer managed	1

2. RDS Permission

created another policy: XYZ-RDS-Management

Used Amazon Q to get JSON script:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowRDSInstanceCreation",
      "Effect": "Allow",
      "Action": [
        "rds:CreateDBInstance",
        "rds:CreateDBCluster",
        "rds:DescribeDBInstances",
        "rds:DescribeDBClusters",
        "rds:DescribeDBSubnetGroups",
        "rds:DescribeDBParameterGroups",
        "rds:DescribeDBClusterParameterGroups",
        "rds:DescribeOptionGroups",
        "rds:ListTagsForResource"
      ],
      "Resource": "*"
    }
  ],
}
```

```

    "Sid": "AllowVPCDescribeForRDS",
    "Effect": "Allow",
    "Action": [
        "ec2:DescribeVpcs",
        "ec2:DescribeSubnets",
        "ec2:DescribeSecurityGroups",
        "ec2:DescribeAvailabilityZones"
    ],
    "Resource": "*"
  },
  {
    "Sid": "AllowIAMRolePassForRDS",
    "Effect": "Allow",
    "Action": [
        "iam:PassRole"
    ],
    "Resource": "*",
    "Condition": {
        "StringEquals": {
            "iam:PassedToService": "rds.amazonaws.com"
        }
    }
  }
]
}

```

Attached XYZ-RDS-Management to group XYZ-EC2-Control.

The screenshot shows the AWS IAM console interface for the 'XYZ-EC2-Control' user group. The 'Permissions' tab is active, displaying a list of three attached policies. Each policy is 'Customer managed' and has 1 attached entity. The policies are 'XYZ-EC2-StartStop', 'XYZ-Network-Management', and 'XYZ-RDS-Management'.

Policy name	Type	Attached entities
XYZ-EC2-StartStop	Customer managed	1
XYZ-Network-Management	Customer managed	1
XYZ-RDS-Management	Customer managed	1

Now the XYZuser has:

EC2

- └─ Start
- └─ Stop

Networking

- └─ Create VPC
- └─ Create Subnet
- └─ Create NACL
- └─ Create Security Group

RDS

- └─ Create/manage DB instance

Task 3: Explore AWS Security Options

1. Enable MFA

IAM → Users → XYZUser → Security credentials → MFA

Configure MFA using an authenticator application.

Step 1

●

Select MFA device

Step 2

○

Set up device

Select MFA device

Info

MFA device name

Device name

This name will be used within the identifying ARN for this device.

MSI-Brave

Maximum 64 characters. Valid characters: A-Z, a-z, 0-9, and + = , . @ _ - (hyphen)

MFA device

Device options

In addition to username and password, you will use this device to authenticate into your account.

○



Passkey or security key

Authenticate using your fingerprint, face, or screen lock. Create a passkey on this device or use another device, like a FIDO2 security key.

AWS recommends using a passkey or security key as an MFA device as they provide the most protection against attacks such as phishing.

○



Authenticator app

Authenticate using a code generated by an app installed on your mobile device or computer.

○



Hardware TOTP token

Authenticate using a code generated by Hardware TOTP token or other hardware devices.

Passkey display name - Optional

This name will be shown when signing in using passkey. The default suggested name can be customized if needed.

●

Use default Passkey display name

430776404757-MSI-Brave-XYZUser

Maximum 64 characters. Valid characters: A-Z, a-z, 0-9, and + = , . @ _ - (hyphen)

Cancel

Next

IAM > IAM users > XYZUser

Passkey MFA device assigned

As a security best practice, we encourage registering multiple devices in the case that your primary method is lost, disabled, or unavailable. Choose any of your MFA devices to use to sign in to your AWS account.

×

Multi-factor authentication (MFA) (1)

Remove

Resync

Assign MFA device

Use MFA to increase the security of your AWS environment. Signing in with MFA requires an authentication code from an MFA device. Each user can have a maximum of 8 MFA devices assigned. [Learn more](#)

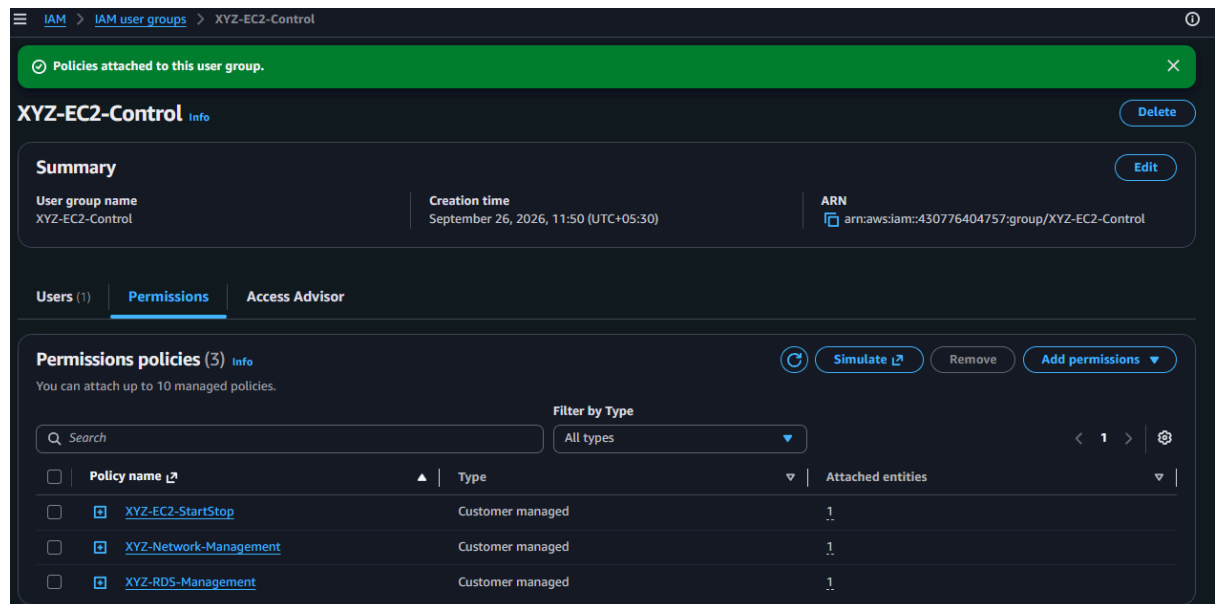
Type	Identifier	Certifications	Created on
○ Passkeys and security keys	arn:aws:iam::430776404757:user/XYZUser/MSI-Brave-LXUJGLIFBEGPPABFQTWNPJWI4	0	Sat Sep 26 2026

2. Followed Least Privilege

1. **Didn't give AdministratorAccess**, unless the person genuinely requires administrator privileges. Instead give only the actions required for the user's job. For example: ec2:StartInstances, ec2:StopInstances rather than: ec2:*

2. Used IAM Policy Conditions

Made permissions more restrictive using conditions. For example, restricted EC2 operations to a particular region or require certain conditions.

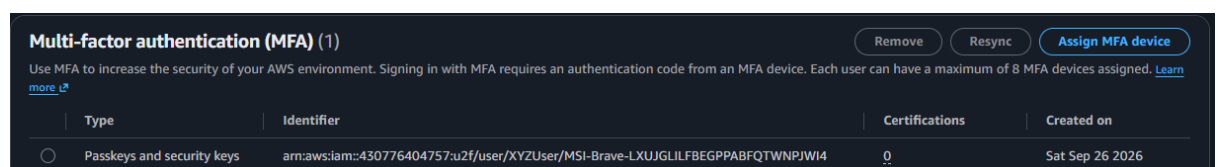


3. Protect the Root Account

The AWS account root user should not be used for normal daily operations.

Recommended controls include:

- Strong root password
- MFA
- No access keys for routine use
- Use IAM identities for administration



4. To Use CloudTrail

Enable/configure: AWS CloudTrail

CloudTrail records API activity such as:

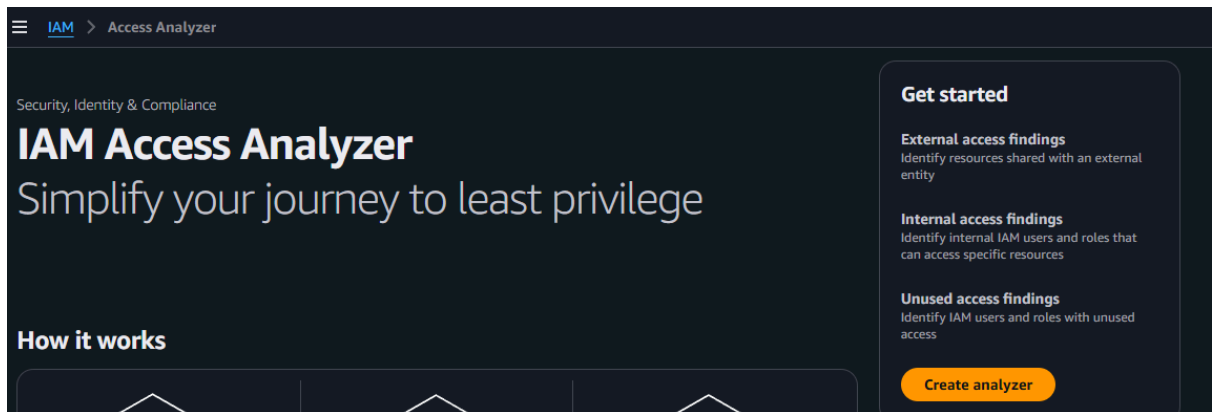
- CreateVpc
- CreateSubnet

- StartInstances
- StopInstances
- CreateDBInstance

This allows administrators to determine who performed an action and when.

5. To use IAM Access Analyzer

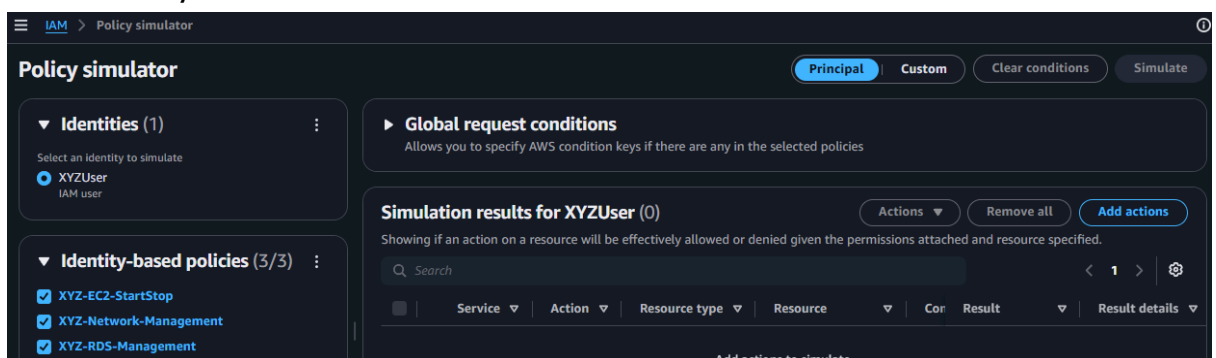
IAM → Access Analyzer



It can help identify resources that are accessible outside the intended trust boundary and assist with reviewing permissions.

6. Use IAM Policy Simulator

IAM → Policy Simulator



This is particularly useful for demonstrating that the permissions are working as intended.

Final Result/Summary

Created an IAM user capable of logging in to the AWS Management Console and created an IAM group with controlled EC2 permissions. The group was granted permission to start and stop EC2 instances. Additional permissions were provided for VPC, subnet, Network ACL, security group and RDS management. AWS security mechanisms including MFA, least-privilege IAM policies, CloudTrail, IAM Access Analyzer and IAM Policy Simulator were explored to improve the security of AWS resources and user permissions.